Image Gallery + Lightbox (Masonry / Pinterest)

- **Problem:** Build a Pinterest-style image gallery. Photos of different heights are packed into a masonry layout, load lazily, and more photos load as the user scrolls. Clicking a photo opens it in a lightbox with Previous / Next, keyboard and swipe navigation.
- **Tech stack:** React (Vite, JavaScript, Tailwind CSS v4)
- **Time limit:** 60 minutes
- **Often asked at:** Pinterest, Meta, Myntra
- **Why it matters:** Layout, lazy loading

PART A: Thought process

General steps for any machine coding task

1. **Understand the requirements.** Restate the problem, split functional vs non-functional, ask clarifying questions, write down assumptions.
2. **List the interactions.** Every user action and its expected result. This is your checklist for building and testing.
3. **Design the architecture.** Name the components and their one-line jobs. Start small.
4. **Design the state.** What is the single source of truth? What can be derived?
5. **Design data structures and contracts.** Shape of the data, API, hook inputs and outputs.
6. **Trace one data flow end to end.** Proves the design works before you type.
7. **Find the hard parts.** Solve them on paper first.
8. **Build in layers.** Simplest working version first, then one working layer at a time.
9. **Cover states and edge cases.** Loading, empty, error, end of list, broken image, resize.
10. **Wrap up.** Trade-offs and what you would add with more time.

Time budget for 60 minutes

Phase	Time
Requirements + clarifying questions	5 min
Interactions, architecture, state, API contract	6 min
Hard parts on paper (masonry math, infinite scroll guard, lightbox focus)	5 min
Build: fetch first page + plain grid	6 min
Build: `computeLayout` + absolute-positioned masonry + resize	10 min
Build: lazy images (reserved box, `loading="lazy"`, `srcset`, fade-in, error)	6 min
Build: infinite scroll (sentinel + guard + dedupe + states)	8 min
Build: lightbox (dialog, prev / next, keys, swipe, preload, focus return)	10 min
Edge cases, accessibility, wrap-up	4 min

Tip: think out loud

Say what you are deciding and why: "I know every photo's width and height from the API, so I can compute each tile's exact position before any image loads. That means no layout jumping, and appending a new page never moves the photos already on screen." The interviewer grades your reasoning as much as the code.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- **Masonry grid:** photos keep their own aspect ratio (no cropping into squares). Each photo goes into the currently **shortest column**, like Pinterest. The number of columns adapts to the screen width.
- **Lazy loading:** images load only when they get near the viewport. Each tile shows a grey placeholder of the right size, then fades the photo in.
- **Infinite scroll:** when the user nears the bottom, the next page of photos loads automatically. Show "Loading…", a Retry on error, and "You've reached the end" when there is nothing more.
- **Lightbox:** clicking a photo opens it full screen with the author and "3 / 120" counter.
  - Previous / Next buttons, Left / Right arrow keys, swipe on touch.
  - Close with the ✕ button, the Esc key, or a click on the dark background.
  - Next keeps working past the last loaded photo (it loads the next page).

Non-functional requirements

Area	What it means here
Performance	No layout shift (CLS = 0): every tile's box is sized before its image loads. Only visible images download. Right-sized images via `srcset`. Layout is O(n · columns) and memoised.
Scalability	Hundreds of photos stay smooth; the layout gives absolute positions, so virtualisation can be added later without changing the math.
Error handling	Failed page request → Retry, photos already shown stay. A broken image → a small fallback in its tile, the grid is not affected.
Accessibility	Tiles are real `<button>`s with alt text, in reading order. The lightbox is a modal dialog: focus moves inside, background is inert, Esc closes, focus returns to the photo you were viewing. Arrow keys work. Counter is announced with `aria-live`.
Usability	Background scroll is locked while the lightbox is open. Neighbour photos are preloaded so Next is instant.
Reusability	`computeLayout` is a pure function; `usePhotos`, `useSentinel`, `useElementWidth` are independent hooks.
Extensibility	Swap the API (Unsplash, own backend) by changing one file; add virtualisation, search or favourites without touching the layout math.

Clarifying questions to ask the interviewer

1. **Do we know each image's width and height before loading it?** (Decides whether we can compute the layout up front or must measure after load.)
2. **Should the order be left-to-right (row reading order) or is shortest-column packing fine?** Pinterest uses shortest column.
3. **Infinite scroll or a "Load more" button?** And is there a real API or should I mock one?
4. **What should the lightbox do at the last photo:** stop, wrap around, or load more?
5. **Which devices?** Do we need touch swipe and different column counts on mobile?
6. **How many photos at most?** (Hundreds is fine in the DOM; tens of thousands needs virtualisation.)

Assumptions we will build with

- Data comes from the public **Picsum API** (`https://picsum.photos/v2/list?page=&limit=`), which returns `width` and `height` for each photo and resizes images on the fly by URL.
- Shortest-column packing, columns at least **220 px** wide, **16 px** gap.
- Infinite scroll with a page size of **30**. A page shorter than 30 means "no more".
- Lightbox stops at both ends (no wrap); near the end it loads the next page.
- Up to a few hundred photos; virtualisation is a "more time" item.

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page opens	First 30 photos are requested; "Loading photos…" shows; grey boxes of the right size appear, then photos fade in
Tall screen, first page does not fill it	The next page loads automatically, without any scroll, until the screen is filled
Scroll near the bottom (800 px before)	Next page loads once, is appended; existing tiles do not move
Fast scroll / two triggers at once	Only one request for the page; no duplicate photos
Page request fails	"Couldn't load photos. Retry"; photos already shown stay
Click Retry	The same page is requested again
Last page received	"You've reached the end."; no more requests
API returns no photos at all	"No photos to show."
Resize the window / rotate phone	Column count and widths recompute; the grid re-packs with no overlap
An image URL fails	That tile shows "Image unavailable · author"; others are unaffected
Tab through the grid	Focus goes photo 1, 2, 3… with a visible outline
Click / Enter on a photo	Lightbox opens on that photo, focus moves to Close, page behind cannot scroll
Next / Right arrow / swipe left	Shows the next photo; counter updates and is announced

Action / input	Expected behavior / output
Prev / Left arrow / swipe right	Shows the previous photo
On the first photo	Prev is disabled; Left arrow does nothing
Within 3 of the last loaded photo	Next page loads in the background; Next stays enabled once it arrives
Esc / ✕ / click on dark area	Lightbox closes; focus returns to the tile of the photo last viewed and it scrolls into view
Click on the photo itself	Nothing (does not close)

---

3. Design the architecture

Module	Responsibility
`api/photos.js`	`fetchPhotos(page, limit, signal)` and `photoUrl(photo, width)`; validates and slims the API data
`utils/masonry.js`	`computeLayout(items, width)`: pure, returns `{ columns, positions[], height }`
`utils/mergeById.js`	Append a page without duplicates
`hooks/usePhotos.js`	Owns `photos` + `status`, `loadMore()` with in-flight guard and abort
`hooks/useSentinel.js`	IntersectionObserver on an empty div at the bottom → calls `loadMore`
`hooks/useElementWidth.js`	ResizeObserver → container width
`components/MasonryGrid.jsx`	Measures width, memoises layout, renders tiles absolutely positioned
`components/PhotoTile.jsx`	One `<li><button><img/></button></li>`: reserved box, lazy image, fade-in, error fallback
`components/Lightbox.jsx`	Modal `<dialog>`: image, counter, prev / next, keys, swipe, preload
`App.jsx`	Wires it together, owns `activeIndex`, status messages, focus return

```
App  (usePhotos → photos, status, loadMore; activeIndex)
├── MasonryGrid  (useElementWidth → width; computeLayout(photos, width))
│   └── PhotoTile × n   (box from layout, onOpen(index))
├── <div sentinel>      (useSentinel(loadMore, enabled: status === 'idle'))
├── <div role="status"> (loading / error + Retry / end / empty)
└── Lightbox            (only when activeIndex !== null)
```

Start simple: first version is App + a plain CSS grid of `<img>`. Split out `MasonryGrid` and `PhotoTile` when the layout math arrives, and `Lightbox` when the click behaviour arrives.

---

4. Design the state / data model

State or field	Purpose
`photos` (in `usePhotos`)	All loaded photos, in API order, no duplicates. The single source of truth
`status` (in `usePhotos`)	`'idle' | 'loading' | 'error' | 'done'`: drives the sentinel and the status text
`nextPage` (ref)	Which page to fetch next; not shown, so a ref
`busy` (ref)	Synchronous in-flight guard against double fetch
`controller` (ref)	`AbortController` of the current request, aborted on unmount
`width` (in `useElementWidth`)	Container width, the only input to the layout besides `photos`
`activeIndex` (in `App`)	Index of the photo open in the lightbox; `null` = closed
`state` (in `PhotoTile`)	`'loading' | 'loaded' | 'error'` for that one image
`image` (in `Lightbox`)	`{ id, status }` of the big image that finished loading

```
// usePhotos
const [photos, setPhotos] = useState([]);
const [status, setStatus] = useState('idle'); // idle | loading | error | done
const nextPage = useRef(1);
const busy = useRef(false);

// App
const [activeIndex, setActiveIndex] = useState(null);

// Lightbox: "is the current photo loaded?" is derived, not reset by an effect
const [image, setImage] = useState({ id: null, status: 'loading' });
const imageStatus = image.id === photo.id ? image.status : 'loading';
```

Senior note: derive, don't store

- **The layout is derived**, never stored: `computeLayout(photos, width)` inside `useMemo`. Storing positions in state would need syncing on every append and resize.
- **Column count, column width, grid height** are all outputs of the same function.
- **`hasPrev` / `hasNext`** come from `index` and `photos.length`.
- **Lightbox loading flag** is derived by comparing ids, so navigating needs no "reset loading" effect (which would flash or race).
- **Store the index, not the photo object** in `activeIndex`: when a new page is appended, `photos[index]` is still right and the counter total updates by itself.
- **Ownership:** `usePhotos` owns the data, `App` owns which photo is open, each tile owns its own load state (no global map of 500 flags).

---

5. Design the data structures and contracts

API contract (Picsum)

```
GET https://picsum.photos/v2/list?page=2&limit=30
```

```
[
  {
    "id": "102",
    "author": "Ben Moore",
    "width": 4320,
    "height": 3240,
    "url": "https://unsplash.com/photos/pJILiyPdrXI",
    "download_url": "https://picsum.photos/id/102/4320/3240"
  }
]
```

- `id` is a **stable string id**: used as the React `key`, for dedupe, and to tie the lightbox image state to a photo.
- `width` / `height` are the original size. We only need the **ratio** `height / width` to size the tile before the image exists.
- Image at any size: `https://picsum.photos/id/{id}/{w}/{h}`. We keep the original ratio, so nothing is cropped.
- End of data: a page with fewer than `limit` items (an empty array past the end).

Our photo model (after `fetchPhotos`)

```
{ id: '102', author: 'Ben Moore', width: 4320, height: 3240 }
```

Slim on purpose: the UI never needs `url` / `download_url`, and items with a zero size are dropped at the boundary (they would cause a division by zero in the layout).

Function / hook contracts

```
fetchPhotos(page, limit, signal)  // -> Promise<Photo[]>, throws on !res.ok, AbortError on abort
photoUrl(photo, width)            // -> 'https://picsum.photos/id/102/640/480'

computeLayout(items, containerWidth, { minColumnWidth = 220, gap = 16 })
// -> { columns: 3, height: 1840,
//      positions: [{ x: 0, y: 0, width: 222.67, height: 167 }, ...] }  // same order as items

mergeById(current, incoming)      // -> new array, or `current` itself if nothing new

usePhotos()                       // -> { photos, status, loadMore }
useSentinel(onVisible, { enabled, rootMargin }) // -> ref for the sentinel div
useElementWidth()                 // -> [ref, width]

<MasonryGrid photos onOpen={(index) => …} />
<Lightbox photos index onIndexChange onClose />
```

---

6. Trace the data flow

Interaction: user scrolls to the bottom, a new page arrives, then they open a new photo.

```
User scrolls; the sentinel div enters the viewport + 800px margin
        |
        ▼
IntersectionObserver callback → isIntersecting → loadMore()
```

```
        │
        ▼
busy.current is false → set true;  setStatus('loading')
        │   (status ≠ 'idle' → useSentinel effect disconnects the observer)
        ▼
fetchPhotos(3, 30, signal) → GET /v2/list?page=3&limit=30
        │
        ▼
200 OK → filter + map to { id, author, width, height }
        │
        ▼
nextPage = 4;  setPhotos(prev => mergeById(prev, page));  setStatus('idle')
        │
        ▼
MasonryGrid re-renders → useMemo: computeLayout(photos, width)
        │   items 0..59 get exactly the same positions as before (prefix-stable);
        │   items 60..89 go into the shortest columns
        ▼
New PhotoTiles mount: grey boxes of the final size; <img loading="lazy">
        │   the browser downloads only the ones near the viewport, picking a
        │   width from srcset; onLoad → state 'loaded' → fade in
        ▼
status 'idle' → useSentinel effect creates a new observer
        │   (if the sentinel is still visible it fires at once → next page)
        ▼
User clicks tile 75 → onOpen(75) → setActiveIndex(75)
        │
        ▼
Lightbox mounts → dialog.showModal() (focus inside, background inert), body scroll locked,
preload photos 74 and 76; 75 ≥ 90 - 3? no → no extra fetch
        │
        ▼
Esc → onCancel → preventDefault → closeLightbox() → setActiveIndex(null)
        → rAF → focus [data-photo-index="75"] (scrolls into view)
```

---

7. Important technical challenges

Challenge 1: Masonry placement without layout jumps

Problem. Photos have different heights. CSS `columns` fills column 1 top to bottom first, so photo 2 appears far down column 1 and, worse, **appending a page re-flows every column**: the photo you were looking at jumps. CSS Grid `masonry` is not supported in all browsers. Measuring each image after it loads causes the grid to jump around as images arrive.

Solution. We know `width` and `height` from the API, so compute the layout **before** any image loads:

1. Column count from the container width: `floor((W + gap) / (minCol + gap))`, at least 1.
2. Keep an array of column heights. Each photo goes into the **shortest** column (leftmost on a tie), its height is `columnWidth × h / w`.
3. Render each tile with `position: absolute; transform: translate(x, y)` and give the container the tallest column height.

Because item *i* only depends on items *0..i-1*, appending a page never changes earlier positions (**prefix-stable**). DOM order stays photo order, so Tab and screen readers go 1, 2, 3.

```
export function computeLayout(items, containerWidth, { minColumnWidth = 220, gap = 16 } = {}) {
  const columns = Math.max(1, Math.floor((containerWidth + gap) / (minColumnWidth + gap)));
  const columnWidth = Math.max(0, (containerWidth - gap * (columns - 1)) / columns);
  const heights = new Array(columns).fill(0);

  const positions = items.map((item) => {
    const column = heights.indexOf(Math.min(...heights)); // leftmost shortest
    const height = Math.round((columnWidth * item.height) / item.width);
    const position = { x: column * (columnWidth + gap), y: heights[column], width: columnWidth, height
};
    heights[column] += height + gap;
    return position;
  });

  return { columns, positions, height: items.length ? Math.max(...heights) - gap : 0 };
}
```

Resizing: a `ResizeObserver` gives the new width, `useMemo` recomputes. O(n · columns), well under a millisecond for hundreds of photos.

Challenge 2: Infinite scroll that never double-fetches and never gets stuck

Problem. Three classic bugs:

- **Double fetch:** the observer fires twice (or the sentinel and the lightbox both ask) before React re-renders with `status = 'loading'` → page 3 is requested twice → duplicate photos and duplicate React keys.
- **Stuck on tall screens:** IntersectionObserver only fires when visibility **changes**. On a 1440 px screen the first 30 photos may not push the sentinel out of view, so it never fires again.
- **StrictMode / unmount:** the request is aborted on unmount; if we treat that as an error, the remounted component shows "Couldn't load" forever.

Solution.

- A **ref** `busy` as a synchronous lock (state updates are not visible in the same tick) + `mergeById` as a second safety net.
- Observe **only when** `status === 'idle'` and rebuild the observer every time it returns to idle. A fresh observer reports the current state immediately, so a still-visible sentinel triggers the next page right away.
- Abort → back to `'idle'`, not `'error'`.
- `rootMargin: '800px'` starts loading before the user reaches the bottom.

```
// usePhotos
const loadMore = useCallback(async () => {
  if (busy.current) return;
  busy.current = true;
  setStatus('loading');
  controller.current = new AbortController();
  try {
    const page = await fetchPhotos(nextPage.current, PAGE_SIZE, controller.current.signal);
    nextPage.current += 1;
    setPhotos((prev) => mergeById(prev, page));
    setStatus(page.length < PAGE_SIZE ? 'done' : 'idle');
  } catch (err) {
```

```
    setStatus(err.name === 'AbortError' ? 'idle' : 'error');
  } finally {
    busy.current = false;
  }
}, []);

// useSentinel
useEffect(() => {
  if (!enabled || !ref.current) return;
  const observer = new IntersectionObserver(([e]) => e.isIntersecting && onVisible(), { rootMargin });
  observer.observe(ref.current);
  return () => observer.disconnect();
}, [enabled, onVisible, rootMargin]);
```

Challenge 3: Lazy, right-sized images with zero layout shift

Problem. Rendering 300 `<img>` tags downloads 300 full-size photos (a 4320 px original is several MB). Without a known box size, each image pushes content down as it arrives (CLS), and with absolute positioning a wrong size means overlapping tiles.

Solution.

- The tile's box gets `width` / `height` from the layout, so the space is **reserved** before the image exists; a grey background is the placeholder.
- Native `loading="lazy"` + `decoding="async"`: the browser loads images near the viewport, no observer per image needed.
- `srcset` with 320 / 640 / 960 widths and `sizes` = the real tile width: the browser picks the smallest file that is sharp on this screen's pixel density. Picsum resizes by URL, keeping the ratio.
- Fade in on `onLoad`; on `onError` show a small fallback inside the same box.

```
<img
  src={photoUrl(photo, 640)}
  srcSet={[320, 640, 960].map((w) => `${photoUrl(photo, w)} ${w}w`).join(', ')}
  sizes={`${Math.round(box.width)}px`}
  alt={`Photo by ${photo.author}`}
  loading="lazy"
  decoding="async"
  onLoad={() => setState('loaded')}
  onError={() => setState('error')}
  className={state === 'loaded' ? 'opacity-100' : 'opacity-0'}
/>
```

Challenge 4: An accessible lightbox (focus, Esc, scroll lock, focus return)

Problem. A `div` overlay lets Tab escape to the page behind, Esc does nothing, screen readers still read the grid, the page scrolls under the overlay, and after closing focus is lost at the top of the page.

Solution. Use the native `<dialog>` + `showModal()`:

- Top layer (no z-index fights), background becomes **inert** (focus trap and hidden from screen readers for free), focus moves to the first button (Close).
- Esc fires `cancel`: we `preventDefault()` and close through React state, so the DOM and state never disagree.

- Lock body scroll while open; `scrollbar-gutter: stable` keeps the grid width from changing when the scrollbar disappears.
- On close, focus the tile of the photo **last viewed** (not the one first clicked), found with `data-photo-index`; `focus()` also scrolls it into view.
- Guard `showModal()` with `if (!dialog.open)`: StrictMode runs effects twice and calling it on an open modal is an error in older browsers.

```
useEffect(() => {
  const dialog = dialogRef.current;
  if (!dialog.open) dialog.showModal();
  document.body.style.overflow = 'hidden';
  return () => { document.body.style.overflow = ''; };
}, []);

<dialog ref={dialogRef} aria-label="Photo viewer"
  onCancel={(e) => { e.preventDefault(); onClose(); }}
  onKeyDown={(e) => { if (e.key === 'ArrowLeft') prev(); if (e.key === 'ArrowRight') next(); }}>

// App
const closeLightbox = () => {
  const last = activeIndex;
  setActiveIndex(null);
  requestAnimationFrame(() => document.querySelector(`[data-photo-index="${last}"]`)?.focus());
};
```

8. Implement in the correct order

#	Step	Working result after this step
1	`fetchPhotos` + `usePhotos` (first page in an effect, no guard yet)	30 photo objects in state
2	Plain CSS grid of `<img src={photoUrl(p, 400)}>`	Something on screen, data proven
3	`computeLayout` (pure) + node asserts	Positions are correct on paper
4	`useElementWidth` + `MasonryGrid` with absolute tiles	Real masonry, re-packs on resize
5	`PhotoTile`: reserved box, `loading="lazy"`, `srcset`, fade-in, error fallback	Fast, no layout shift
6	`useSentinel` + `loadMore` with `busy` ref + `mergeById`	Infinite scroll, no duplicates
7	Status line: loading / error + Retry / end / empty; abort on unmount	Every list state handled
8	`Lightbox` with `<dialog>`: image, counter, Close, Esc, backdrop click	Opens and closes correctly
9	Prev / Next buttons, arrow keys, swipe, preload neighbours, image loading / error	Full navigation
10	Load more near the end from the lightbox; focus return; scroll lock	Polished, accessible

9. Cover all states and edge cases

State / edge case	What the user sees / what the code does
Initial (before width is measured)	Empty grid; tiles render only when `width > 0` (avoids a 0-width flash)
Loading first page	"Loading photos…" below the empty grid
Success	Masonry grid; tiles grey until each image fades in
Empty (API returns `[]` on page 1)	"No photos to show."
Error on a page	"Couldn't load photos. Retry"; loaded photos stay; sentinel off until Retry
End of data	"You've reached the end."; observer off for good
Tall screen, first page too short	Fresh observer fires at once → pages load until the screen is filled
Double trigger (scroll + lightbox)	`busy` ref ignores the second call
Overlapping pages / duplicate ids	`mergeById` skips them, keys stay unique
Unmount mid-request (or StrictMode)	Request aborted; status back to `'idle'`, no stuck error
Bad API item (width or height 0)	Dropped in `fetchPhotos`, layout never divides by 0
Very narrow screen (< 220 px)	1 column of the full width
Resize / rotate	New width → new layout; same columns until a breakpoint
Broken image URL in grid	"Image unavailable · author" inside the same box
Broken image in lightbox	"This photo could not be loaded."; Prev / Next still work
Slow big image in lightbox	"Loading…" text, previous image not shown under the new counter
First / last photo in lightbox	Prev / Next disabled; arrow key is a no-op
Near the end in lightbox	Next page loaded in the background, Next re-enables
Esc / ✕ / dark-area click	Close; focus returns to the last viewed tile
Click on the image itself	Does not close
Lightbox open, user scrolls wheel	Page does not scroll (body overflow hidden)
Reduced motion	Hover zoom only with `motion-safe:`

---

10. Final structure

```
_70_image_gallery_lightbox_masonry/
├── index.html
├── package.json
├── vite.config.js
└── src/
    ├── main.jsx
    ├── index.css              tailwind + scrollbar-gutter
    ├── App.jsx                wiring, activeIndex, status line, focus return
    ├── api/
    │   └── photos.js          fetchPhotos, photoUrl
```

```
    ├── utils/
    │   ├── masonry.js          computeLayout (pure)
    │   └── mergeById.js        dedupe append (pure)
    ├── hooks/
    │   ├── usePhotos.js        photos, status, loadMore
    │   ├── useSentinel.js      IntersectionObserver trigger
    │   └── useElementWidth.js  ResizeObserver width
    └── components/
        ├── MasonryGrid.jsx     layout + absolute tiles
        ├── PhotoTile.jsx       lazy image, placeholder, error
        └── Lightbox.jsx        <dialog>, nav, keys, swipe, preload
```

---

11. Full working code

`vite.config.js`

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react-swc';
import tailwindcss from '@tailwindcss/vite';

export default defineConfig({
  plugins: [react(), tailwindcss()],
});
```

`src/index.css`

```
@import "tailwindcss";

/* Reserve the scrollbar space so locking body scroll (lightbox) does not change the grid width. */
html {
  scrollbar-gutter: stable;
}
```

`src/main.jsx`

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import './index.css';
import App from './App';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

`src/api/photos.js`

```
const BASE = 'https://picsum.photos';

// GET /v2/list?page=1&limit=30 -> [{ id, author, width, height, url, download_url }]
export async function fetchPhotos(page, limit, signal) {
  const res = await fetch(`${BASE}/v2/list?page=${page}&limit=${limit}`, { signal });
  if (!res.ok) throw new Error(`Request failed: ${res.status}`);
```

```
  const data = await res.json();
  // Keep only what the UI needs; drop items without a usable size (they would break the layout math).
  return data
    .filter((p) => p.width > 0 && p.height > 0)
    .map(({ id, author, width, height }) => ({ id, author, width, height }));
}

// Picsum resizes on the fly: /id/{id}/{w}/{h}. Same ratio as the original, so no cropping.
export const photoUrl = (photo, width) =>
  `${BASE}/id/${photo.id}/${width}/${Math.round((width * photo.height) / photo.width)}`;
```

src/utils/masonry.js

```
// Greedy "shortest column" placement. Item i only depends on items 0..i-1,
// so appending a page never moves the photos already on screen.
export function computeLayout(items, containerWidth, { minColumnWidth = 220, gap = 16 } = {}) {
  const columns = Math.max(1, Math.floor((containerWidth + gap) / (minColumnWidth + gap)));
  const columnWidth = Math.max(0, (containerWidth - gap * (columns - 1)) / columns);
  const heights = new Array(columns).fill(0);

  const positions = items.map((item) => {
    const column = heights.indexOf(Math.min(...heights)); // leftmost shortest column
    const height = Math.round((columnWidth * item.height) / item.width);
    const position = { x: column * (columnWidth + gap), y: heights[column], width: columnWidth, height
};
    heights[column] += height + gap;
    return position;
  });

  return { columns, positions, height: items.length ? Math.max(...heights) - gap : 0 };
}
```

src/utils/mergeById.js

```
// Append a page, skipping ids we already have (pages can overlap if the list shifts).
export function mergeById(current, incoming) {
  const seen = new Set(current.map((item) => item.id));
  const fresh = incoming.filter((item) => {
    if (seen.has(item.id)) return false;
    seen.add(item.id);
    return true;
  });
  return fresh.length ? [...current, ...fresh] : current;
}
```

src/hooks/usePhotos.js

```
import { useCallback, useEffect, useRef, useState } from 'react';
import { fetchPhotos } from '../api/photos';
import { mergeById } from '../utils/mergeById';

const PAGE_SIZE = 30;

// status: 'idle' (can load more) | 'loading' | 'error' | 'done' (no more pages)
export function usePhotos() {
  const [photos, setPhotos] = useState([]);
  const [status, setStatus] = useState('idle');
```

```
  const nextPage = useRef(1);
  const busy = useRef(false); // sync guard: two triggers in the same tick must not fetch the same
page
  const controller = useRef(null);

  const loadMore = useCallback(async () => {
    if (busy.current) return;
    busy.current = true;
    setStatus('loading');
    controller.current = new AbortController();
    try {
      const page = await fetchPhotos(nextPage.current, PAGE_SIZE, controller.current.signal);
      nextPage.current += 1;
      setPhotos((prev) => mergeById(prev, page));
      setStatus(page.length < PAGE_SIZE ? 'done' : 'idle');
    } catch (err) {
      // An aborted request is not an error: go back to idle so the next mount can retry.
      setStatus(err.name === 'AbortError' ? 'idle' : 'error');
    } finally {
      busy.current = false;
    }
  }, []);

  useEffect(() => () => controller.current?.abort(), []);

  return { photos, status, loadMore };
}
```

src/hooks/useSentinel.js

```
import { useEffect, useRef } from 'react';

// Calls onVisible when the sentinel element comes near the viewport.
// The observer is rebuilt every time `enabled` turns true; a new observer reports the
// current state right away, so if the sentinel is still visible after a page loads
// (tall screen, few photos) the next page is requested without any scroll.
export function useSentinel(onVisible, { enabled, rootMargin = '800px' }) {
  const ref = useRef(null);

  useEffect(() => {
    const el = ref.current;
    if (!enabled || !el) return;
    const observer = new IntersectionObserver(
      ([entry]) => entry.isIntersecting && onVisible(),
      { rootMargin }
    );
    observer.observe(el);
    return () => observer.disconnect();
  }, [enabled, onVisible, rootMargin]);

  return ref;
}
```

src/hooks/useElementWidth.js

```
import { useEffect, useRef, useState } from 'react';

// Width of an element, kept current by a ResizeObserver (fires once on observe, then on every
```

```
resize).
export function useElementWidth() {
  const ref = useRef(null);
  const [width, setWidth] = useState(0);

  useEffect(() => {
    const observer = new ResizeObserver(([entry]) => setWidth(entry.contentRect.width));
    observer.observe(ref.current);
    return () => observer.disconnect();
  }, []);

  return [ref, width];
}
```

`src/components/PhotoTile.jsx`

```
import { useState } from 'react';
import { photoUrl } from '../api/photos';

const WIDTHS = [320, 640, 960];

export default function PhotoTile({ photo, index, box, onOpen }) {
  const [state, setState] = useState('loading'); // loading | loaded | error

  return (
    // The box size comes from the aspect ratio, so it is reserved before the image arrives (no layout
shift).
    <li
      className="absolute top-0 left-0"
      style={{ transform: `translate(${box.x}px, ${box.y}px)`, width: box.width, height: box.height }}
    >
      <button
        type="button"
        data-photo-index={index}
        onClick={() => onOpen(index)}
        className="group block h-full w-full overflow-hidden rounded-xl bg-slate-200 focus-
visible:outline-3 focus-visible:outline-offset-2 focus-visible:outline-blue-600"
      >
        {state === 'error' ? (
          <span className="flex h-full items-center justify-center p-2 text-xs text-slate-500">
            Image unavailable · {photo.author}
          </span>
        ) : (
          <img
            src={photoUrl(photo, 640)}
            srcSet={WIDTHS.map((w) => `${photoUrl(photo, w)} ${w}w`).join(', ')}
            sizes={`${Math.round(box.width)}px`}
            alt={`Photo by ${photo.author}`}
            loading="lazy"
            decoding="async"
            onLoad={() => setState('loaded')}
            onError={() => setState('error')}
            className={`h-full w-full object-cover transition duration-300 motion-safe:group-
hover:scale-105 ${
              state === 'loaded' ? 'opacity-100' : 'opacity-0'
            }`}
          />
        )}
      </button>
```

```
    </li>
  );
}
```

`src/components/MasonryGrid.jsx`

```
import { useMemo } from 'react';
import { useElementWidth } from '../hooks/useElementWidth';
import { computeLayout } from '../utils/masonry';
import PhotoTile from './PhotoTile';

export default function MasonryGrid({ photos, onOpen }) {
  const [ref, width] = useElementWidth();
  const layout = useMemo(() => computeLayout(photos, width), [photos, width]);

  return (
    // DOM order = photo order, so Tab and screen readers follow 1, 2, 3... not column by column.
    <ul ref={ref} aria-label="Photos" className="relative" style={{ height: layout.height }}>
      {width > 0 &&
        photos.map((photo, i) => (
          <PhotoTile key={photo.id} photo={photo} index={i} box={layout.positions[i]} onOpen={onOpen}
/>
        ))}
    </ul>
  );
}
```

`src/components/Lightbox.jsx`

```
import { useEffect, useRef, useState } from 'react';
import { photoUrl } from '../api/photos';

const fullUrl = (photo) => photoUrl(photo, Math.min(photo.width, 1600));
const SWIPE_PX = 50;

export default function Lightbox({ photos, index, onIndexChange, onClose }) {
  const dialogRef = useRef(null);
  const touchStartX = useRef(0);
  const [image, setImage] = useState({ id: null, status: 'loading' });

  const photo = photos[index];
  const imageStatus = image.id === photo.id ? image.status : 'loading'; // derived: no reset effect on
navigation
  const hasPrev = index > 0;
  const hasNext = index < photos.length - 1;
  const prev = () => hasPrev && onIndexChange(index - 1);
  const next = () => hasNext && onIndexChange(index + 1);

  // showModal(): top layer, background made inert (focus trap for free), focus moved inside.
  useEffect(() => {
    const dialog = dialogRef.current;
    if (!dialog.open) dialog.showModal(); // guard: StrictMode runs this effect twice
    document.body.style.overflow = 'hidden';
    return () => {
      document.body.style.overflow = '';
    };
  }, []);
```

```
  // Warm the cache so Next / Prev feel instant.
  useEffect(() => {
    [photos[index - 1], photos[index + 1]].forEach((p) => {
      if (p) new Image().src = fullUrl(p);
    });
  }, [photos, index]);

  const onKeyDown = (e) => {
    if (e.key === 'ArrowLeft') prev();
    if (e.key === 'ArrowRight') next();
  };

  const onTouchEnd = (e) => {
    const dx = e.changedTouches[0].clientX - touchStartX.current;
    if (dx > SWIPE_PX) prev();
    if (dx < -SWIPE_PX) next();
  };

  const navButton = 'absolute top-1/2 -translate-y-1/2 rounded-full bg-white/10 px-4 py-3 text-3xl
hover:bg-white/20 disabled:opacity-30 focus-visible:outline-2 focus-visible:outline-white';

  return (
    <dialog
      ref={dialogRef}
      aria-label="Photo viewer"
      onCancel={(e) => {
        e.preventDefault(); // Esc: let React state close it, so state and DOM never disagree
        onClose();
      }}
      onKeyDown={onKeyDown}
      className="m-0 h-dvh max-h-none w-screen max-w-none bg-black/90 p-0 text-white backdrop:bg-
black/60"
    >
      <div
        className="flex h-full flex-col"
        onTouchStart={(e) => (touchStartX.current = e.touches[0].clientX)}
        onTouchEnd={onTouchEnd}
      >
        <div className="flex items-center justify-between gap-4 p-4">
          <p aria-live="polite" className="text-sm">
            {index + 1} / {photos.length} · {photo.author}
          </p>
          <button
            type="button"
            onClick={onClose}
            aria-label="Close"
            className="rounded-full px-3 py-1 text-2xl hover:bg-white/20 focus-visible:outline-2
focus-visible:outline-white"
          >
            ×
          </button>
        </div>

        {/* Click on the dark area (not the image or buttons) closes. */}
        <div
          className="relative flex min-h-0 flex-1 items-center justify-center px-4 pb-6 sm:px-20"
          onClick={(e) => e.target === e.currentTarget && onClose()}
        >
          {imageStatus === 'loading' && <span className="absolute text-sm text-white/70">Loading…
</span>}
```

```
          {imageStatus === 'error' ? (
            <p className="text-sm text-white/70">This photo could not be loaded.</p>
          ) : (
            <img
              key={photo.id}
              src={fullUrl(photo)}
              alt={`Photo by ${photo.author}`}
              onLoad={() => setImage({ id: photo.id, status: 'loaded' })}
              onError={() => setImage({ id: photo.id, status: 'error' })}
              className={`max-h-full max-w-full object-contain transition-opacity ${
                imageStatus === 'loaded' ? 'opacity-100' : 'opacity-0'
              }`}
            />
          )}
          <button type="button" onClick={prev} disabled={!hasPrev} aria-label="Previous photo"
className={`${navButton} left-4`}>
            ‹
          </button>
          <button type="button" onClick={next} disabled={!hasNext} aria-label="Next photo" className=
{`${navButton} right-4`}>
            ›
          </button>
        </div>
      </div>
    </dialog>
  );
}
```

src/App.jsx

```
import { useEffect, useState } from 'react';
import Lightbox from './components/Lightbox';
import MasonryGrid from './components/MasonryGrid';
import { usePhotos } from './hooks/usePhotos';
import { useSentinel } from './hooks/useSentinel';

export default function App() {
  const { photos, status, loadMore } = usePhotos();
  const [activeIndex, setActiveIndex] = useState(null); // null = lightbox closed
  const sentinelRef = useSentinel(loadMore, { enabled: status === 'idle' });

  // Keep the lightbox fed: near the last loaded photo, fetch the next page so Next keeps working.
  useEffect(() => {
    if (activeIndex !== null && activeIndex >= photos.length - 3 && status === 'idle') loadMore();
  }, [activeIndex, photos.length, status, loadMore]);

  const closeLightbox = () => {
    const last = activeIndex;
    setActiveIndex(null);
    // Return focus to the photo the user ended on (focus() also scrolls it into view).
    requestAnimationFrame(() => document.querySelector(`[data-photo-index="${last}"]`)?.focus());
  };

  return (
    <main className="mx-auto max-w-7xl px-4 py-6">
      <header className="mb-6 flex items-baseline justify-between">
        <h1 className="text-2xl font-bold text-slate-800">Gallery</h1>
        <p className="text-sm text-slate-500">{photos.length} photos</p>
      </header>
```

```
      <MasonryGrid photos={photos} onOpen={setActiveIndex} />

      <div ref={sentinelRef} className="h-px" aria-hidden="true" />
      <div role="status" className="py-8 text-center text-sm text-slate-500">
        {status === 'loading' && 'Loading photos…'}
        {status === 'error' && (
          <>
            Couldn't load photos.{' '}
            <button type="button" onClick={loadMore} className="font-medium text-blue-600 underline">
              Retry
            </button>
          </>
        )}
        {status === 'done' && (photos.length ? "You've reached the end." : 'No photos to show.')}
      </div>

      {activeIndex !== null && (
        <Lightbox photos={photos} index={activeIndex} onIndexChange={setActiveIndex} onClose=
{closeLightbox} />
      )}
    </main>
  );
}
```

---

12. Wrap-up

Edge cases handled

- Layout computed from known aspect ratios: zero layout shift, no overlap, appended pages never move existing photos.
- Responsive column count via ResizeObserver; 1 column on tiny screens; bad sizes filtered at the API boundary.
- Lazy, right-sized images (`loading="lazy"` , `srcset` / `sizes`), placeholder, fade-in, per-tile error fallback.
- Infinite scroll with a synchronous guard, dedupe, abort on unmount, auto-fill on tall screens, Retry and end-of-list.
- Lightbox: native modal dialog (focus trap, inert background, Esc), arrow keys, swipe, preloading, loading / error for the big image, scroll lock, loads more near the end, returns focus to the last viewed photo.

Trade-offs to mention

- **JS absolute layout vs CSS `columns` / CSS Grid masonry:** CSS `columns` needs no JS but orders top-to-bottom and re-flows on append; native grid masonry is not in all browsers yet. JS layout needs the aspect ratios up front and re-runs on resize, but gives stable, Pinterest-like order and is ready for virtualisation.
- **Absolute positions vs flex columns:** rendering N flex columns is simpler CSS, but the DOM order becomes column-by-column, which breaks Tab order and screen-reader order. Absolute positioning keeps DOM order = photo order.
- **Native `loading="lazy"` vs an IntersectionObserver per image:** native is zero code and the browser tunes the distance; a custom observer gives exact control (e.g. a blur-up placeholder swap) at the cost

of more code.
- **Infinite scroll vs "Load more" / pagination:** infinite scroll is natural for browsing but makes the footer unreachable and is hard to deep-link; a button is simpler and more accessible.
- `<dialog>` **vs a custom portal modal:** `<dialog>` gives focus trap, inert and Esc for free; a custom modal needs a focus-trap library and `aria-hidden` on siblings, but allows enter / exit animations more easily.
- **No tile memoisation:** each append re-renders all tiles. Fine for hundreds; with thousands, virtualise first (it removes far more work than `memo`).

What to add with more time

- **Tests:** unit tests for `computeLayout` and `mergeById` (pure); React Testing Library for the lightbox (open, arrows, Esc, focus return) with a mocked `fetch` and IntersectionObserver.
- **Virtualisation:** render only tiles whose `y` range overlaps the viewport ± 1 screen; positions are already known, so it is a filter over `layout.positions`.
- **Placeholders:** a dominant colour or BlurHash from the API instead of plain grey.
- **Deep links:** `?photo=102` in the URL so a lightbox can be shared and the Back button closes it.
- **Enter / exit animations** and pinch-zoom in the lightbox; a "View original" link.
- **Restore scroll position** when coming back to the gallery (cache pages in memory or sessionStorage).

---

13. Counter questions the interviewer may ask

Requirements and approach

Q1. Why not just use CSS `column-count`?

It packs top-to-bottom: photo 2 ends up far down column 1, not next to photo 1. And when a new page is appended, the browser rebalances every column, so photos the user is looking at jump. It is a fine answer for a static gallery, not for infinite scroll.

Q2. What about CSS Grid `masonry` / `grid-template-rows: masonry`?

It is the future answer and needs no JS, but it is not supported in all major browsers yet. I would use it behind `@supports` once it ships everywhere; until then, the JS layout is predictable everywhere.

Q3. What if the API does not give image dimensions?

Then we cannot place a tile before its image loads. Options: ask the backend to add them (best, it is cheap at upload time); or place tiles as images load (`onLoad` gives `naturalWidth/Height`), appending to the shortest column in arrival order, which means a less stable order; or use a fixed ratio with `object-cover` (crops).

State design

Q4. Why store `activeIndex` instead of the selected photo object?

The photo is derived: `photos[activeIndex]`. When a page is appended while the lightbox is open, the index is still correct and the "3 / 120" total updates for free. Storing the object would duplicate data that can go stale.

Q5. Why are `busy` and `nextPage` refs and not state?

They are not shown on screen, and `busy` must change **synchronously**: two calls in the same tick would both read the old state value and both fetch. A ref is updated immediately. The visible `status` is still state, so the UI re-renders.

Q6. Why is the layout not stored in state?

It is fully determined by `photos` and `width`. `useMemo(() => computeLayout(photos, width), [photos, width])` recomputes only when either changes. Storing it would mean keeping two sources of truth in sync on every append and resize.

Masonry layout

Q7. Why does appending a page not move the existing photos?

The algorithm is greedy and only looks backwards: the column for photo *i* depends only on photos *0..i-1*. Recomputing with more items gives the exact same positions for the first ones. Only a width change (a new column count) re-packs everything.

Q8. How do you pick the number of columns?

`floor((width + gap) / (minColumnWidth + gap))`, at least 1. Adding `gap` once on the top accounts for n columns having only n-1 gaps. Then the columns stretch to fill the width exactly. Breakpoint-free and works for any container, not just the window.

Q9. The window resizes 60 times a second while dragging. Is that a problem?

ResizeObserver delivers at most once per frame, and the layout is O(n · columns): for 500 photos that is a few thousand operations, well under a millisecond. If it ever became heavy: throttle with `requestAnimationFrame`, or only recompute when the **column count** changes and scale x / width in between.

Q10. Why `transform: translate()` and not `top` / `left`?

Both work. `transform` is composited by the GPU, so if we animate tiles to their new place on resize (a common follow-up), it is smooth and does not trigger layout for every frame.

Infinite scroll

Q11. Why IntersectionObserver and not a scroll listener?

A scroll listener fires dozens of times per second on the main thread and needs `getBoundingClientRect` (forces layout) plus throttling. IntersectionObserver is computed by the browser off the main thread and fires only when visibility changes.

Q12. What if the API is slow, and the user keeps scrolling?

`busy` blocks further requests, "Loading photos…" shows at the bottom, and the 800 px `rootMargin` means we usually start before the user gets there. For very slow APIs: fetch two pages ahead, or show skeleton tiles with estimated heights.

Q13. What if the user scrolls fast and the sentinel fires twice?

The second call sees `busy.current === true` and returns. Even if something slipped through, `mergeById` drops duplicate ids, so React keys stay unique.

Q14. What if there are 1 million photos?

The fetching part is fine (pages). The DOM is not: 1M tiles would freeze the page. Add **windowing**: since every position is known, render only tiles whose `y` overlaps `[scrollY - screen, scrollY + 2 · screen]`:

```
const visible = layout.positions
  .map((box, i) => ({ box, i }))
  .filter(({ box }) => box.y + box.height > top && box.y < bottom);
```

Also cap memory by dropping far-away pages and keeping only their heights, and binary-search per column instead of a linear filter.

Lazy loading

Q15. Why native `loading="lazy"` instead of your own observer?

It is zero code, works without JS-driven state per image, and the browser picks the distance based on network speed. I would write my own observer only for effects the browser cannot do, like swapping a blurred placeholder for the real image.

Q16. What do `srcset` and `sizes` buy us here?

The tile is ~220-400 CSS px wide; on a 2× screen that needs ~800 px, not the 4320 px original. `sizes` tells the browser the real display width, and it picks the smallest `srcset` candidate that is sharp enough, often 10× fewer bytes.

Q17. How do you avoid layout shift (CLS)?

The tile's width and height come from the layout before the image loads, so the image has nowhere to push. Same idea as setting `width` / `height` attributes or `aspect-ratio` on images in a normal flow.

Lightbox

Q18. Why `<dialog>` and not a div with `position: fixed`?

`showModal()` gives, for free: the top layer (no z-index war), an inert background (Tab cannot leave, screen readers ignore the page), and Esc via the `cancel` event. With a div I would have to build a focus trap and set `aria-hidden` / `inert` on everything else myself.

Q19. Why `preventDefault()` on `cancel` and close through state?

If the browser closed the dialog itself, the DOM would say "closed" while React still has `activeIndex` set, and the component would stay mounted. Letting React decide keeps one source of truth: state → `null` → Lightbox unmounts.

Q20. Why return focus to the last viewed photo, not the clicked one?

The user may have pressed Next 20 times. Returning them to where they started loses their place; returning to the photo they ended on (and `focus()` scrolls to it) matches what they were looking at, which is also what Pinterest and Google Photos do.

Q21. How do you make Next feel instant?

Preload neighbours with `new Image().src = url` whenever the index changes, so the browser cache already has them. Also use the same URL builder (same size) for preload and display, otherwise the cache misses.

Q22. What happens if the user presses Next on the last loaded photo?

Next is disabled at the very end, but when the user is within 3 photos of the end, App calls `loadMore()`. When the page arrives `photos.length` grows, `hasNext` becomes true and Next re-enables, without closing the lightbox.

Trade-offs and "why not X?"

Q23. Why not a library like `react-masonry-css` or `react-window`?

In an interview they want to see the algorithm; the core is 15 lines. In production I would consider a library for virtualised masonry (e.g. `masonic`) if we really need tens of thousands of items, but not just for placement.

Q24. How would you test this?

`computeLayout` and `mergeById` are pure: unit tests for column count, shortest-column choice, prefix stability, empty input. Components: React Testing Library with a mocked `fetch`, a fake IntersectionObserver that I trigger manually, and assertions for open → ArrowRight → Esc → focused tile.

Q25. How would you support a URL like `/photo/102` that opens the lightbox directly?

Keep the photo id in the URL (`?photo=102`), open the lightbox from it on load (fetch that photo if it is not in the list yet), push history on open so the Back button closes the lightbox. The index stays derived: `photos.findIndex(p => p.id === id)`.